Методичка по TDD

ТЕОРИЯ: ЧТО ТАКОЕ TDD? 2

ПРАКТИЧЕСКАЯ ЧАСТЬ 3

Теория: что такое TDD?

TDD (Test-Driven Development или разработка через тестирование) — это методология создания программного обеспечения, при которой автоматический тест пишется *до* того, как будет написан сам программный код.

Классический цикл TDD: «Красный — Зеленый — Рефакторинг»

Процесс строится на повторении коротких шагов (обычно занимающих от нескольких минут):

1. **Красный (Red):** Пишется минимальный автоматический тест для еще не существующей функции. Тест запускается и ожидаемо завершается с ошибкой (ведь самого кода нет).
2. **Зеленый (Green):** Пишется самый простой и быстрый рабочий код, который заставляет этот тест пройти успешно. На этом этапе код может быть не идеальным.
3. **Рефакторинг (Refactor):** Код очищается от дублирования и оптимизируется, при этом тесты постоянно запускаются заново, чтобы убедиться, что ничего не сломалось. (Здесь тесты уже подключаем к методам, которые уже используется в приложении).

Главные преимущества TDD

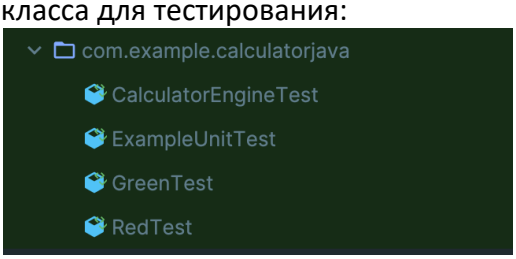
- **Высокое качество кода:** Изначально создается протестированный и понятный дизайн системы.
- **Защита от регрессий:** Наличие набора тестов позволяет безопасно менять код в будущем — любая поломка сразу подсвечивается тестом.
- **Простота поддержки:** Код пишется с учетом того, как его будут проверять, поэтому он получается более модульным.

Основные недостатки и критика

- **Временные затраты на старте:** Написание тестов до кода замедляет первую фазу разработки.
- **Порог входа:** Требует дисциплины и опыта в проектировании архитектуры и написании юнит-тестов.

Практическая часть

За основу мы возьмем проект на Java, в папке проекта com.example.calculatorjava(Test) и создаем в нем три java класса для тестирования:



RedTest – для написания первого теста без метода
GreenTest – тест с методом заглушкой
CalculatorEngineTest – тестирование основного метода который мы используем в калькуляторе
Важно: для удобства методы и классы для калькулятора будут созданы в одном файле с тестом (вам нужно будет использовать основной метод, из проекта, который создан в отдельном файле)
По циклу разработки TDD пропишем РЕД тест. Пропишите код теста:

RedTest — Java

```java
package com.example.calculatorjava;

//Библиотека JUnit (класс Assert). Статический импорт метода assertEquals.
//Благодаря этому можно писать assertEquals(...) вместо Assert.assertEquals(...).
import static org.junit.Assert.assertEquals;
//Импорт аннотации @Test из JUnit 4. Помечает метод как тестовый.
import org.junit.Test;
//"red" — красная фаза, когда тест падает, потому что функционала ещё нет
public class RedTest {
//Вложенный класс
    static class Calculator {
        //Текущее содержимое "дисплея" калькулятора. Инициализируется строкой "0".
        private String input = "0";
//Флаг: нужно ли сбросить ввод перед следующей цифрой
        private boolean resetInputOnNextDigit = false;
//Обработка нажатия цифровой клавиши (0–9)
    //Если стоит флаг сброса — начинаем ввод заново с нуля и снимаем флаг.
    //если на дисплее ровно "0" — заменяем на введённую цифру (чтобы не было "05", а было "5");
    //иначе — конкатенируем (input + d — int автоматически превращается в строку).
        public void digit(int d) {
            if (resetInputOnNextDigit) {
                input = "0";
                resetInputOnNextDigit = false;
            }
            input = input.equals("0") ? String.valueOf(d) : input + d;
        }
//Заглушка (пустая реализация) — обработчик нажатия =.
// Пока ничего не делает, но нужна, чтобы тест компилировался.
        public void equalsPressed() { }
//Возвращает текущее содержимое дисплея — то, что тест будет проверять через assertEquals.

        public String getDisplay() {
            return input;
        }
        // метод add() пока не написан
    //именно поэтому тест сейчас в "красной" фазе TDD.
    }
//Аннотация @Test — JUnit запустит этот метод как тест.
// Имя по соглашению: <что>_<условия>_<ожидание>.
```

class Calculator – тестовый класс с методами ввода, вывода и вычисления значений

Дальше реализуем GreenTest:

GreenTest Java

```java
package com.example.calculatorjava;

import static org.junit.Assert.assertEquals;

import org.junit.Test;
//реализация пока фейковая
public class GreenTest {

    static class Calculator {
        //Храним первый операнд или можем накопленный результат хранить после выполнения
        //то, что было на дисплее в момент нажатия +.
        // исапользуем double, чтобы позже работать с дробями.
        private double accumulator = 0;
        //Флаг: «есть ли отложенная операция сложения».
        // true — пользователь нажал +, но ещё не нажал =.
        // Нужен, чтобы понять, в каком состоянии калькулятор.
        private boolean pendingAdd = false;
        //Флаг: «следующая цифра должна начать новый ввод с нуля».
        // Устанавливается в true после + и после =.
        // Отличается от pendingAdd: например, после = операция уже не «в ожидании»,
        // но новую цифру всё равно надо начинать с чистого листа.
        private boolean resetInputOnNextDigit = false;
        private String input = "0";

        public void digit(int d) {
            if (resetInputOnNextDigit) {
                input = "0";
                resetInputOnNextDigit = false;
            }
            input = input.equals("0") ? String.valueOf(d) : input + d;
        }
//Превращает строку с дисплея в число
        private double parseInput() {
            return Double.parseDouble(input);
        }

        // Заглушка (фейк метод): возвращаем константу, лишь бы тест прошёл
        //минимальное изменение, чтобы тест стал зелёным.
        private void evaluatePending() {
            input = "5";
        }
```

GreenTest Java

```java
//if (pendingAdd && !resetInputOnNextDigit) — операция ожидается и второе число
// действительно введено.
// Только тогда вычисляем. Если нажать = 2 + = сразу после + (без второго числа)
// — ничего не считаем.
//pendingAdd = false — после = операция завершена.
//resetInputOnNextDigit = true — следующая цифра начнёт новое выражение с чистого листа
// (как в реальном калькуляторе после =).
        public void equalsPressed() {
            if (pendingAdd && !resetInputOnNextDigit) {
                evaluatePending();
            }
            pendingAdd = false;
            resetInputOnNextDigit = true;
        }
```

GreenTest Java

```java
        public String getDisplay() {
            return input;
        }
    }

    @Test
    public void add_twoPlusThree_displaysFive() {
        Calculator c = new Calculator();
        c.digit(2);
        c.add();
        c.digit(3);
        c.equalsPressed();
        assertEquals("5", c.getDisplay()); // проходит
    }
}
```

В этом файле создаем метод add с тестовыми значениями как заглушка, чтобы наши тесты не падали и успешно проходили

Дальше реализуем Рефактор этап (тут мы будем уже использовать наш основной класс для вычислений):

Refactore Java

```java
package com.example.calculatorjava;

import static org.junit.Assert.assertEquals;

import org.junit.Before; //аннотация JUnit 4: метод с ней запускается перед каждым тестом
import org.junit.Test;

public class CalculatorEngineTest {
//Поле для тестируемого объекта — пересоздаётся перед каждым тестом,
// чтобы тесты не влияли друг на друга.
    private CalculatorEngine engine;
//Создаёт свежий экземпляр калькулятора перед каждым тестом — изоляция тестов
    @Before
    public void setUp() {
        engine = new CalculatorEngine();
    }

    @Test
    public void ad_krut() {
        engine.inputDigit(2);
        engine.add();
        engine.inputDigit(3);
        engine.calculate();

        assertEquals("5", engine.getDisplay());
        assertEquals("2 + 3 =", engine.getExpression());
    }

    @Test
    public void add_mnogo_zifr() {
        engine.inputDigit(1);
        engine.inputDigit(2);
        engine.inputDigit(0);
        engine.add();
        engine.inputDigit(8);
        engine.inputDigit(0);
        engine.calculate();

        assertEquals("200", engine.getDisplay());
    }
```

Refactore Java

```java
    @Test
      public void add_zero() {
          engine.inputDigit(0);
          engine.inputDigit(1);
          engine.add();
          engine.inputDigit(0);
          engine.inputDigit(2);
          engine.calculate();

          assertEquals("3", engine.getDisplay());
      }

      @Test
      public void add_resultZero() {
          engine.inputDigit(0);
          engine.add();
          engine.inputDigit(0);
          engine.calculate();

          assertEquals("0", engine.getDisplay());
      }

      @Test
      public void rez() {
          engine.inputDigit(1);
          engine.add();
          engine.inputDigit(2);
          engine.add();                  // 1 + 2 вычисляется при втором «+»

          assertEquals("3", engine.getDisplay());
          assertEquals("3 +", engine.getExpression());

          engine.inputDigit(3);
          engine.calculate();

          assertEquals("6", engine.getDisplay());
          assertEquals("3 + 3 =", engine.getExpression());
      }

      @Test
      public void dobavlenie_s_sumoi() {
          engine.inputDigit(2);
          engine.add();
          engine.inputDigit(3);
          engine.calculate();           // 5

          engine.add();
          engine.inputDigit(4);
          engine.calculate();

          assertEquals("9", engine.getDisplay());
          assertEquals("5 + 4 =", engine.getExpression());
      }

      @Test
      public void dobavlenie_so_znakom() {
          engine.inputDigit(7);
          engine.add();

          assertEquals("7 +", engine.getExpression());
          assertEquals("7", engine.getDisplay());
      }

      @Test
      public void calculation() {
          engine.inputDigit(9);
          engine.calculate();

          assertEquals("9", engine.getDisplay());
          assertEquals("", engine.getExpression());
      }
}
```

В этом файле мы уже проводим все возможные тесты для метода сложения
Аннотацией @Before мы помечаем, то что будет запускаться перед тестом в нем мы инициализируем объект класса CalculatorEngine. И дальше под аннотацией @Test мы проводим наши тесты.

Самостоятельная работа

1. Реализовать по методу TDD оставшиеся методы для калькулятора
2. Показать и объяснить на паре